一. 单选题（共 8 题）

1

【单选题】动态规划算法与分治法类似，其基本思想也是将待求解问题分解成若干个子问题，主要差别（）。

A、子问题是否容易计算

B、问题能否递归解决

C、子问题是否相互独立

D、是自顶向下还是自底向上解决问题

正确答案：C

2

【单选题】证明问题具有最优子结构性质最常见的方法是（）。

A、枚举法 B、数学归纳法 C、反证法 D、应用主定理

正确答案：C

3

【单选题】应用动态规划法解决问题的基本活动如下：

(E)找出最优解的性质

(D)刻划其结构特征。

(C)递归地定义最优值。

(B)以自底向上的方式计算出最优值。

(A)根据计算最优值时得到的信息构造最优解。

正确的步骤排序是（）。

A、(A)(B)(C)(D)(E)

B、(E)(D)(B)(A)(C)

C、(D)(E)(C)(B)(A)

D、(E)(D)(C)(B)(A)

E、(E)(C)(D)(A)(B)

正确答案：D

4

【单选题】通常不同的子问题个数随问题的大小呈（）增长。因此用动态规划法只需要非指数或非阶乘的时间，从而获得较高的解题效率。

A、常量级　B、线性级　C、多项式级 D、平方级

正确答案：C

5【单选题】n个矩阵连乘积问题共有（）个不同的子问题。

A、$O(n^2)$

B、$\Omega^{(n^2)}$

C、$\Theta^{(n^2)}$

D、都不对

正确答案：C

6【单选题】计算T(n)

$$T(1) \geq 1,$$

$$T(n) \geq 1 + \sum_{k=1}^{n-1}(T(k) + T(n-k) + 1) \qquad \text{for } n > 1$$

A、 $\Theta^{(n^2)}$

B、 $\Omega^{(n^2)}$

C、 $O^{(2^n)}$

D、 $\Omega^{(2^n)}$

E、 $\Theta^{(2^n)}$

**我的答案：** D

**正确答案：** D

**答案解析：** 用替换法

7【单选题】给定2个序列X={$x_1$,$x_2$,...,$x_m$}和Y={$y_1$,$y_2$,...,$y_n$}，找出X和Y的最长公共子序列。该问题具有（）个不同的子问题。

A、 O(m+n)

B、 Ω(m+n)

C、 Θ(m+n)

D、 Ω(mn)

E、 Θ(mn)

**我的答案：** E

**正确答案：** E

8【单选题】**游艇出租问题**：

游艇俱乐部沿江设置了n个游艇出租站1,2,…n，游客可在这些游艇出租站租用游艇，并在下游的任何一个游艇出租站归还游艇。游艇出租站i到游艇出租站j之间的租金为r(i,j)，1<=i<j<=n，设计一个算法，计算出从出租站1到出租站n所需要的最少租金。
下面说法不正确的是（）。

A、 求解这个问题不能采用折半策略。

B、 如果得到了从站点1到n-1站点的最少租金方案，就可以求解从站点1到n的最少租金。

C、 选择从站点1选择最少租金的站点k，继续进行，直到站点n。

D、 这些说法都不正确。

**我的答案：** C

**正确答案：** C

✓ **3.1** 分

## 二. 填空题（共10题）

9【填空题】递归算法求解问题时，每次产生的子问题并不总是新问题，有些子问题被反复计算多次，这种性质称为子问题的（）性质。

**我的答案：** 3.1 分

第一空：
重叠

**正确答案：**

第一空：
重叠

10【填空题】问题的（）性质是该问题可用动态规划算法和贪心算法求解的关键特征。

**我的答案：** 3.1 分

第一空：
最优子结构

**正确答案：**

第一空：
最优子结构

11【填空题】
有一个问题用动态规划法求解，其复杂度公式是 $T(n)=1+\sum_{j=0}^{j=n-1}T(j), T(0)=1$ ，求解T(n)=（）。

**我的答案：** 3.1 分

第一空：
$2^n$

**正确答案：**

第一空：
$2^n$

12【填空题】自顶向下的（）法也可以实现动态规划算法，其控制结构与直接递归方法的控制结构相同，这个方法为每个解过的子问题用表格保存起来在需要时查看，避免了相同子问题的重复求解。

**我的答案：** 3.1 分

第一空：
备忘录

**正确答案：**

第一空：
备忘录

13【填空题】给定一个无序的整数序列*a*[0..*n*-1]，求其中最长递增子序列的长度。
例如，*a*[]={2，1，5，3，6，4，8，9，7}，*n*=9，其最长递增子序列为{1，3，4，8，9}，结果为5。
设一维数组dp[0..n-1]，dp[*i*]表示*a*[0..*i*]中以*a*[*i*]结尾的最长递增子序列的长度。
算法结束后，dp[0..n-1]的值是（）。

**我的答案：** 3.1 分

第一空：
{1,1,2,2,3,3,4,5,4}

**正确答案：**

第一空：
{1, 1, 2, 2, 3, 3, 4, 5, 4 }

14【填空题】有一个行数n和列数m的矩阵，n≥1, m≥1它每个格子有一个权值。从左上角的格子开始每次只能向右或者向下走，最后到达右下角的位置，路径上所有的数字累加起来就是路径和，求所有的路径中最小的路径和。
设dp[n][m]为走到n*m位置的路径长度，那么递推公式dp[n][m] =

**我的答案：** 3.1 分

第一空：
min{dp[n-1][m],dp[n][m-1]},n>1,m>1

**正确答案：**

第一空：
min(dp[n-1][m],dp[n][m-1]), n>1, m>1

第 14 题答案改为 min{dp[n-1][m],dp[n][m-1]}+1

15【填空题】游艇出租问题：

游艇俱乐部沿江设置了n个游艇出租站1,2,…n，游客可在这些游艇出租站租用游艇，并在下游的任何一个游艇出租站归还游艇。游艇出租站i到游艇出租站j之间的租金为r(i,j)，1<=i<j<=n，设计一个算法，计算出从出租站1到出租站n所需要的最少租金。
可以用动态规划求解该问题。设dp[n][n]表示从站点i到站点j的最少租金，关键是找出dp的递推关系。分析下面代码，应该填写什么。

```
for (int r = 2; r <= n; r++) {
      for (int i = 1; i <= n - r + 1; i++) {
         int j = i + r - 1;
         for (int k = i; k <= j; k++)  {
            int t =  () ;
            if (t < dp[i][j]) {
               dp[i][j] = t;//用较优解替换原来的r(i,j)
            }
      }
}
```

**我的答案：** 3.1 分

第一空：
dp[i][k]+dp[k][j]

**正确答案：**

第一空：
dp[i][k] + dp[k][j]

16【填空题】游艇出租问题：
游艇俱乐部沿江设置了n个游艇出租站1,2,...n，游客可在这些游艇出租站租用游艇，并在下游的任何一个游艇出租站归还游艇。游艇出租站i到游艇出租站j之间的租金为r(i,j)，1<=i<j<=n，设计一个算法，计算出从出租站1到出租站n所需要的最少租金。
可以用动态规划求解该问题。
令dp[i]表示从站点1到站点i所需的最少租金。下面的递推关系中空白处（下划线）应该填什么。
dp[1]=r[1][1]=0,
dp[2]=r[1][2],
递推关系：dp[i]=min(dp[i],___ ), 1<j<i

**我的答案：** 3.1 分

第一空：
dp[j]+r[j][i]

**正确答案：**

第一空：
dp[j]+r[j][i]

17【填空题】根据一个动态规划问题的递推公式：

$$P(i,j)=\begin{cases}1 & i=0,\ j>0\\ 0 & i>0,\ j=0,\\ (P(i-1,j)+P(i,j-1))/2, & i>0,\ j>0,\ i+j=n\end{cases}$$

你得到的该问题时间复杂性的O值是什么？

**我的答案：**

第一空：
$O(n^2)$

**正确答案：**

第一空：
$O(2^n)$

3.1 分

18【填空题】把长度为k>0的圆木截成整数长度的数段，使其乘积最大。这个问题可以用动态规划求解。用dp[i]表示截取长度为i时最大乘积，求解得代码如下：

```
int maxProduct(int k) {
    int[] dp = new int[k+1];
    dp[0]=0;
    dp[1]=1;
    int bestP;
    int remaining;
    int currentP;
    for(int i = 2; i <= k; i++) {
        bestP = i;
        for(int cut = 2; cut < (i/2+1); cut++) {
            remaining = i-cut;
            currentP = dp[cut]*dp[remaining];
            if(currentP > bestP) {
                bestP=currentP;
            }
        }
        dp[i] = bestP;
    }
    return dp[k];
}
```

求解得关键是递推关系，可以看出：对长度为k的圆木如果截取长度c，剩余长度是k-c，则dp[k]就是最大值MAX()，c的取值是2到k的一半。

**我的答案：**

第一空：
c,dp[c]*dp[k-c]

**正确答案：**

第一空：
c, dp(c)dp(k-c)

3.1 分

## 三. 多选题（共9题）

19【多选题】一个问题可以采用动态规划法解决，通常要（）。

A、 具有最优子结构性质。

B、 具有递归性质。

C、 所分解出的各个子问题是相互独立的。

D、 各个子问题容易求解。

**我的答案：** ABD

**正确答案：** ABD

3.1 分

20【多选题】应用动态规划法解决问题，通常包含的动作有（）。

A、 找出最优解的性质

B、 递归地定义最优值

C、 以自顶向下的方式计算出最优值。

D、 刻划最优子结构特征

E、 递归地构造最优解

**我的答案：** ABDE

**正确答案：** ABDE

3.1 分

21【多选题】给定一个无序的整数序列*a*[0..*n*-1]，求其中最长递增子序列的长度。
例如，*a*[]={3,1,5,2,6,4,8,10,7, 9, 15, 12}，*n*=12，其最长递增子序列为{1，2，4，8，10，15}，结果为6。
这个问题可以用动态规划法解决。
设一维数组dp[0..n-1]，dp[*i*]表示*a*[0..*i*]中以*a*[*i*]结尾的最长递增子序列的长度。建立dp[*i*]的递推公式，初始化正确的是（）。

A、 d[0]=0

B、 d[1]=1

C、 d[2]=1

D、 d[2]=2

**我的答案：** BC

**正确答案：** BC

**答案解析：** 每个整数都至少是自己的最长递增子序列，长度=1

✓ **3.1** 分

22【多选题】给定一个无序的整数序列*a*[0..*n*-1]，求其中最长递增子序列的长度。
例如，*a*[]={3,1,5,2,6,4,8,10,7, 9, 15, 12}，*n*=12，其最长递增子序列为{1，2，4，8，10，15}，结果为6。
这个问题可以用动态规划法解决。设一维数组dp[0..n-1]，dp[*i*]表示*a*[0..*i*]中以*a*[*i*]结尾的最长递增子序列的长度。
建立dp[*i*]的递推公式，dp[*i*]=max(dp[*i*]，dp[*j*]+1)，条件包括（）。

A、 $a[i] \geq a[j]$

B、 $0 \leq i \leq n-1$

C、 $j \leq i$

D、 $0 \leq j$

**我的答案：** BD

**正确答案：** BD

**答案解析：** a[i] > a[j], j < i,

✓ **3.1** 分

23【多选题】 有一个行数n和列数m的矩阵，它每个格子有一个权值。从左上角的格子开始每次只能向右或者向下走，最后到达右下角的位置，路径上所有的数字累加起来就是路径和，求所有的路径中最小的路径和。

A、 这个问题具有最优子结构

B、 这个问题的子问题有重叠

C、 这个问题的子问题相互独立

D、 这个问题满足贪心选择性质

E、 这个问题可以采用递归法求解

**我的答案：** ABE

**正确答案：** ABE

✓ **3.1** 分

24【多选题】有一个行数n和列数m的矩阵grid，每个格子有一个权值。从左上角的格子开始每次只能向右或者向下走，最后到达右下角的位置，路径上所有的数字累加起来就是路径长度，求所有的路径中最小的路径长度。

例如，

| | | | |
|---|---|---|---|
| 1 | 2 | -3 | 5 |
| 3 | -1 | 4 | -2 |
| -1 | 2 | 5 | -4 |

结果是-2，路径是黄色标记的格子：

| | | | |
|---|---|---|---|
| 1 | 2 | -3 | 5 |
| 3 | -1 | 4 | -2 |
| -1 | 2 | 5 | -4 |

这个问题具有下列性质。

A、 最优子结构性质

B、 贪心选择性质

C、 子问题重叠

D、 子问题独立

**我的答案：** AC

**正确答案：** AC

3.1 分

25【多选题】有一个行数n和列数m的矩阵grid，每个格子有一个权值。从左上角的格子开始每次只能向右或者向下走，最后到达右下角的位置，路径上所有的数字累加起来就是路径长度，求所有的路径中最小的路径长度。

设dp[i][j]为走到i*j位置的路径长度，正确的递推关系有（）。

A、 dp[1][1]=grid[1][1]

B、 $dp[1][j]=\sum_{k=1}^{j} grid[1][k], 1 \le j \le m$

C、 $dp[j][1]=\sum_{k=1}^{j} grid[k][1], 1 \le j \le n$

D、 $dp[i][j]=dp[i][j-1]+dp[i-1][j]+grod[i][j], 1<i \le n, 1<j \le m$

**我的答案：** ABC

**正确答案：** ABC

**答案解析：** dp[n][m] =
min(dp[n-1][m],dp[n][m-1])+ grid[n] [m],

3.2 分

26【多选题】设A、B两组进行对抗比赛，对于确定的n，哪组先胜n局则判哪组取胜。假设A、B能力相当，每组取胜一局的概率为1/2。用P(i. j)表示A组需要胜利i局才能取胜，B组需要胜利j局才能取胜，求解在这种情况下A组取胜的概率。
例如，n=4，A组已胜利2局，B组胜利1局，则i=2，j=3，此时A组取胜的概率为P(2,3)。要求计算出P(i. j)的值。
正确说法的有（）。

A、 若i=0，j>0，则A组已经取胜n局，即P(0, j)=1；

B、 若i>0，j=0，则B组已经取胜n局，即P(i, 0)=0；

C、 若i>0，j>0，P(i, j)应当为P(i, j) 和P(i,j-1)的平均值。

D、 若i>0，j>0，P(i, j)应当为P(i-1, j) 和P(i,j1)的平均值。

E、 若i>0，j>0，P(i, j)应当为P(i-1, j) 和P(i,j-1)的平均值。

**我的答案：** ABE

**正确答案：** ABE

✓ **3.2** 分

27【多选题】把长度为k>0的圆木截成整数长度的数段，使其乘积最大。这个问题可以用动态规划求解。用dp[i]表示截取长度为i时最大乘积，求解得代码如下：

```
1  int maxProduct(int k) {
2      int[] dp = new int[k+1];
3      dp[0]=0;
4      dp[1]=1;
5      int bestP;
6      int remaining;
7      int currentP;
8      for(int i = 2; i <= k; i++) {
9          bestP = i;
10         for(int cut = 2; cut < (i/2+1); cut++) {
11             remaining = i-cut;
12             currentP = dp[cut]*dp[remaining];
13             if(currentP > bestP) {
14                 bestP=currentP;
15             }
16         }
17         dp[i] = bestP;
18     }
19     return dp[k];
20 }
```

关于这个问题正确的说法有（）。

A、 dp[0]=0; // 长度为0 时截取之积也是0

B、 dp[1]=1; // 长度为1时不能截取

C、 代码第10行表示圆木截取最大只能到一半

D、 代码第10-17行表示截取圆木的长度范围是$\left[2..\left[\frac{k}{2}\right]\right]$

**我的答案：** ABD

**正确答案：** ABD

✓ **3.2** 分

## 四. 判断题（共5题）

28【判断题】同一个问题只有一种方式刻划它的最优子结构，使得求解速度更快、空间占用小，问题的维度低。

**我的答案：** 错

**正确答案：** 错

**答案解析：** 多种方式

✓ **3.2** 分

29【判断题】动态规划法通常利用问题的最优子结构性质，以自顶向下的方式递归地从子问题的最优解逐步构造出整个问题的最优解。

**我的答案：** 错

**正确答案：** 错

✓ **3.2** 分

30【判断题】不同的子问题个数随问题的大小呈多项式增长，因此用动态规划算法总可以用多项式时间解决问题，从而获得较高的解题效率。

**我的答案：** 错

**正确答案：** 错

**答案解析：** 可以，但不总能

✓ **3.2** 分

31【判断题】一般而言，当一个问题的所有子问题都至少要解一次时，用动态规划算法比用备忘录方法好。

**我的答案：** 对

**正确答案：** 对

✓ **3.2** 分

32【判断题】有一个行数n和列数m的矩阵grid，每个格子有一个权值。从左上角的格子开始每次只能向右或者向下走，最后到达右下角的位置，路径上所有的数字累加起来就是路径长度，求所有的路径中最小的路径长度。

例如

| | | | |
|---|---|---|---|
| 1 | 2 | -3 | 5 |
| 3 | -1 | 4 | -2 |
| -1 | 2 | 5 | -4 |

结果的值是-2，路径是用黄色标记

| | | | |
|---|---|---|---|
| 1 | 2 | -3 | 5 |
| 3 | -1 | 4 | -2 |
| -1 | 2 | 5 | -4 |

研究这个路径发现，用贪心选择策略就可以得到相同的结果：每次从当前格子的右边和下边寻找权值最小的格子。

这个问题真的可以用贪心算法解决吗?

**我的答案：** 错

**正确答案：** 错

**答案解析：** 找一个反例就行

✓ **3.2** 分